

Beginner Guide to the Three Core Files

A plain-language explanation of the API, IBM Bob agents, and workflow controller - written for presenters who do not read code every day.

Files covered: `app/main.py`, `.bob/custom_modes.yaml`, `workflow/handoff_controller.py`

1. First, the big picture

You do not need to memorize every line of code. Think of the project as a small company with three important documents. `app/main.py` is the product itself: the to-do API. `.bob/custom_modes.yaml` is the job-description and access-control document for the six IBM Bob agents. `workflow/handoff_controller.py` is the supervisor who checks whether the right evidence exists before allowing the next stage.

File	Simple analogy	Main purpose
<code>app/main.py</code>	The restaurant kitchen	Receives API requests, validates them, saves todos, and returns answers.
<code>.bob/custom_modes.yaml</code>	Employee job descriptions plus keycard access	Defines what every AI agent may do and which files it may edit.
<code>workflow/handoff_controller.py</code>	A quality-control receptionist	Checks evidence, requires human approval, records the hand-off, and tells the team which Bob mode is next.

The information flow is: a requirement becomes a design brief; code is built; code is reviewed; independent tests are run; failures are diagnosed; and documentation is produced only after a passing result.

A tiny glossary

Term	Plain meaning
API	A way for one program, website, or tool to ask another program to perform an action.
Endpoint	One API address for one action, such as <code>POST /todos</code> to create a todo.
Database	A structured place to save information. Here it is a local SQLite file.
Schema	The expected shape and rules for incoming or outgoing data.
HTTP status code	A number that tells the caller whether a request worked: 200 means success, 201 means created, 404 means not found, and 422 means invalid input.
Agent	A specialized IBM Bob mode that has a specific role, instructions, and permitted tools.

2. `app/main.py` - the running application

This is the complete implementation of the to-do REST API. When you run `'python -m uvicorn app.main:app --reload'`, Uvicorn starts the FastAPI application object named `app` from this file. The browser Swagger page at `/docs` is generated from the routes and data models defined here.

Part A: imports

```
import sqlite3
from contextlib import asynccontextmanager
from pathlib import Path
from typing import Optional

from fastapi import FastAPI, HTTPException, Response, status
from pydantic import BaseModel, ConfigDict, Field, StrictBool, field_validator
```

Import	Why it exists
sqlite3	Lets Python create, read, update, and delete records in SQLite.
Path	Builds the database file path safely beside main.py.
Optional	Lets update fields be omitted.
FastAPI	Creates the web API and route decorators.
HTTPException / Response / status	Returns correct errors, empty responses, and named HTTP status codes.
Pydantic classes	Describes and validates request and response data.

Part B: where data is saved

```
DB_PATH = str(Path(__file__).parent / "todos.db")
```

`__file__` means this Python file. `Path(__file__).parent` means the app folder. The code creates the path `app/todos.db`. Using this path keeps the data file next to the application instead of somewhere unpredictable on the computer.

Part C: data rules, called Pydantic models

```
class TodoCreate(BaseModel):
    title: str = Field(max_length=200)

    @field_validator("title")
    @classmethod
    def title_not_empty(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("title must not be empty")
        return v
```

`TodoCreate` describes the JSON body expected by `POST /todos`. It requires title to be text. `Field(max_length=200)` prevents titles longer than 200 characters. `v.strip()` removes spaces at both ends, so a title containing only spaces becomes empty and is rejected. If validation fails, FastAPI automatically returns HTTP 422.

TodoUpdate and TodoOut

```
class TodoUpdate(BaseModel):
    title: Optional[str] = Field(default=None, max_length=200)
    done: Optional[StrictBool] = None

class TodoOut(BaseModel):
    id: int
    title: str
    done: bool
```

`TodoUpdate` is used by `PUT`. `Optional` means title or done may be absent. `StrictBool` is important: it accepts JSON true or false, not a loose value such as the text 'yes'. `TodoOut` is the response shape. It guarantees callers

receive id, title, and done with the expected types.

3. app/main.py - database setup and helpers

Creating the table once

```
def _init_db(path: str = DB_PATH) -> None:
    conn = sqlite3.connect(path)
    try:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS todos (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                title TEXT NOT NULL,
                done INTEGER NOT NULL DEFAULT 0
            )
        """)
        conn.commit()
    finally:
        conn.close()
```

`_init_db` means 'initialize database'. `sqlite3.connect` opens the database file. `CREATE TABLE IF NOT EXISTS` creates the table the first time but does nothing if it already exists, so the app can safely start many times. `id` is automatically created by SQLite. `done` is stored as 0 or 1 because SQLite does not have a separate Boolean type. `commit` saves the change. `finally` always closes the database connection, even if an error happens.

Opening a connection for a route

```
def _get_conn(path: Optional[str] = None) -> sqlite3.Connection:
    conn = sqlite3.connect(path or DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn
```

Each API request needs a database connection. `path` or `DB_PATH` means: use a special path if one was provided; otherwise use the normal app database. Tests provide a temporary database path. `sqlite3.Row` lets the code use column names like `row['done']` instead of remembering number positions.

Starting FastAPI

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    _init_db(DB_PATH)
    yield
```

```
app = FastAPI(title="Todo API", lifespan=lifespan)
```

A decorator beginning with `@` adds behavior to the function below it. `lifespan` runs at application startup. It creates the database table, then `yield` means 'the app may now run'. FastAPI uses this lifecycle function every time the server starts.

4. app/main.py - every API route

Route	What user does	What code does
POST /todos	Creates a task	Validates title, inserts a database row, then returns the newly created todo with status 201.
GET /todos	Lists tasks	Reads every row and returns a JSON array.
GET /todos/{id}	Reads one task	Looks up the ID. If no row exists, raises a 404 error.
PUT /todos/{id}	Updates fields	Updates title, done, or both. An empty JSON body is treated as a no-op.
PATCH /todos/{id}/toggle	Flips completion	Reads current done state, changes 0 to 1 or 1 to 0, then returns the new value.
DELETE /todos/{id}	Deletes task	First checks the row exists, deletes it, then returns 204 No Content.

POST /todos explained

```
@app.post("/todos", response_model=TodoOut, status_code=status.HTTP_201_CREATED)
def create_todo(payload: TodoCreate):
    conn = _get_conn()
    try:
        cursor = conn.execute(
            "INSERT INTO todos (title) VALUES (?)", (payload.title,)
        )
        conn.commit()
        row = conn.execute(
            "SELECT id, title, done FROM todos WHERE id = ?", (cursor.lastrowid,)
        ).fetchone()
    finally:
        conn.close()
    return dict(row)
```

- @app.post connects this Python function to the POST /todos URL.
- payload: TodoCreate means FastAPI validates the incoming JSON before the function runs.
- The ? is a parameter placeholder. The user title is provided separately in (payload.title,), which is safer than joining text into SQL.
- cursor.lastrowid is the ID SQLite generated for the new todo.
- The route reads that row back and returns it as JSON. response_model=TodoOut converts done from SQLite's 0/1 into a Boolean.

GET, PUT, PATCH, and DELETE

GET routes use SELECT statements and return existing rows. GET /todos/{id} checks whether fetchone() returned nothing; if it did, it raises HTTPException(404, 'todo not found'). PUT checks which fields were supplied, then chooses one of three fixed UPDATE statements. PATCH toggle reads the current done value and writes the opposite value. DELETE first checks that the ID exists, deletes it, commits, and returns a Response with HTTP 204 NO CONTENT.

Important judge answer: The code uses separate fixed UPDATE statements instead of dynamically building SQL column names. This was a review-driven safety improvement.

5. .bob/custom_modes.yaml - IBM Bob's agent definitions

YAML is a human-readable configuration format. This file is not application code. It tells IBM Bob what custom modes exist, what each mode's job is, when to use it, and which capabilities it is allowed to use. The important idea is least privilege: every agent gets only the permissions needed for its job.

How one mode is structured

```
- slug: coding-agent
  name: Coding Agent
  roleDefinition: ...
  whenToUse: ...
  customInstructions: ...
  groups:
    - read
    - - edit
      - fileRegex: "^app/.*"
    - execute
```

YAML field	Plain-language meaning
slug	Internal short identifier for the mode.
name	Label that users see in IBM Bob.
roleDefinition	The agent identity and its core responsibility.
whenToUse	When the team should select this mode.
customInstructions	Detailed task rules, expected output file, and prohibitions.
groups	Allowed tool capabilities such as read, edit, execute, skill, or subtask.
fileRegex	A pattern that limits which file paths an agent can edit.

The six agents in simple words

Agent	Reads / thinks about	Can write	Cannot do
Research	Scope and requirements	design_brief.md	Cannot write application code.
Coding	Design brief and failure diagnosis	app/	Cannot write review/test reports outside app.
Review	Code, scope, and design brief	review_report.md	Cannot alter the app or tests.
Testing	Code and design brief	app/test_*.py and test_report.md	Cannot modify application source.
Debug	Failed tests, code, and brief	debug_report.md	Cannot fix code.
Pitch/Docs	Verified artifacts	app/README.md and docs/demo-script.md	Cannot change application behavior.

Why fileRegex is powerful

```
Coding Agent: ^app/.*$
Testing Agent: ^(app/test_.*\.py|test_report\.md)$
Review Agent: ^review_report\.md$
```

^ means the beginning of a file path. .* means any remaining characters. \$ means the end of the path. So ^app/.* means any file inside app/. The Testing pattern permits only test files that start with test_ inside app/ plus the root-level test report. It does not match app/main.py, so Testing Agent cannot edit the production API.

Judge-safe explanation: 'The permissions are not only written as a request to the agent. IBM Bob checks the configured file scope before an edit is allowed.'

6. workflow/handoff_controller.py - workflow governance

This file is ordinary Python, not IBM Bob configuration. It adds a controlled hand-off process around the manual mode selection. It is deliberately honest: it does not control or impersonate Bob. A person still chooses the next mode in Bob. The controller checks evidence and records the decision.

The allowed stages

```
STAGES = ("research", "coding", "review", "testing", "debug", "docs")
```

This tuple is simply a fixed list of valid stage names. It prevents typing a random or misspelled stage into the command.

Gate: one rule for one hand-off

```
@dataclass(frozen=True)
class Gate:
    source: str
    target: str
    check: Callable[[Path], tuple[bool, str]]
    input_files: tuple[str, ...]
    output_file: str
    bob_mode: str
```

A dataclass is a convenient way to keep related data together. One Gate describes a valid hand-off. source and target are the two stages. check is a function that returns either true or false plus a readable reason. input_files tells the team what the next agent should read. output_file tells them what it should create. bob_mode is the exact IBM Bob mode that must be selected.

Evidence checks

```
def _review_approved(root: Path) -> tuple[bool, str]:
    report = _read(root, "review_report.md")
    passed = "## Verdict: APPROVED" in report
    return passed, "Review verdict is APPROVED" if passed else "Review report is not APPROVED"
```

This function reads review_report.md. The word 'in' checks whether the approval text exists. It returns a pair: true plus a success message when approved, or false plus an explanation when not approved. Similar functions check whether tests passed or failed.

The configured workflow gates

```
Gate("review", "testing", _review_approved, ... , "Testing Agent")
Gate("testing", "docs", _tests_passed, ... , "Pitch/Docs Agent")
Gate("testing", "debug", _tests_failed, ... , "Debug Agent")
```

These lines express the core governance rules. Review to Testing requires an APPROVED report. Testing to Docs requires PASS and zero failures. Testing to Debug requires a FAIL report. An invalid route, such as Research directly to Docs, is blocked because it has no gate.

7. Controller commands and the audit log

Status command

```
python workflow/handoff_controller.py status
```

This generates docs/workflow/status.md. It is a snapshot that shows each stage as READY or BLOCKED and explains the evidence behind the result.

Handoff command

```
python workflow/handoff_controller.py handoff review testing --approved-by "Presenter Name"
```

The command checks the requested gate, requires a name after --approved-by, and only then writes an audit record. If the review report is not approved, it prints a Blocked message and does not create the hand-off.

What is written to JSONL

```
{
  "timestamp_utc": "...",
  "source": "review",
  "target": "testing",
  "approved_by": "Presenter Name",
  "evidence": "Review verdict is APPROVED",
  "bob_mode": "Testing Agent"
}
```

JSONL means JSON Lines: one JSON record per line. This makes the log easy to append to and easy for a future dashboard or audit program to read. The controller does not edit app/ source, so it cannot bypass the agent permission model.

8. How all three files work together

Question	Which file answers it?	Simple answer
What does the application do?	app/main.py	It offers six validated API operations for a local SQLite to-do list.
Who is allowed to change which file?	.bob/custom_modes.yaml	IBM Bob custom modes use role instructions and fileRegex edit limits.
When may the workflow advance?	workflow/handoff_controller.py	Only when required evidence is present and a named human approves the valid next step.

Example success path: Research Agent writes design_brief.md. Coding Agent builds app/main.py. Review Agent writes an APPROVED report. The controller permits Testing Agent. Testing Agent writes tests and a PASS report. The controller permits Pitch/Docs Agent. At each point, the next role reads the evidence from the prior role.

Example failure path: Testing finds a problem and writes FAIL. The controller then permits Debug Agent. Debug Agent writes a root-cause report, not a code fix. The controller permits Coding Agent, which applies only the diagnosed repair. Testing runs again to verify the fix.

9. Questions you can now answer

Q: Why is title validation in a model?

A: It keeps validation at the API boundary. Invalid data is rejected before it reaches the database.

Q: Why are SQL values written with a question mark?

A: The question mark is a safe placeholder. SQLite receives user values separately from SQL instructions, reducing injection risk.

Q: Why does done use 0 and 1 in SQLite?

A: SQLite does not have a separate Boolean storage class. The API converts that database representation into true and false for users.

Q: Why is Testing Agent not allowed to edit app/main.py?

A: Independent testing is meaningful only if the testing role cannot quietly change the code it is judging.

Q: Can the controller automatically run IBM Bob?

A: No. It validates workflow evidence and records the human decision. A person still selects the next Bob custom mode.

Q: Why use a report file at every stage?

A: Reports make the reasoning visible, reusable by the next role, and auditable by humans.

Q: Why is Debug Agent separate from Coding Agent?

A: It forces a diagnosis before a repair, reducing random or unrelated code changes after a test failure.

Q: What is the most important limitation?

A: The demo is local and deliberately small. A production system would need authentication, stronger audit protections, PostgreSQL, monitoring, and deployment controls.

10. Study plan before judging

- Read this guide once without opening the code.
- Then open app/main.py and connect its sections to this guide: models, database helpers, startup, routes.
- Open .bob/custom_modes.yaml and read only the Coding, Testing, and Review agents first; then read the remaining roles.
- Open workflow/handoff_controller.py and trace only status -> evidence check -> handoff -> JSONL log.
- Practice explaining one success path and one failure path in your own words.
- Do not try to memorize syntax. Memorize the purpose, boundary, evidence, and reason for each part.

One final sentence to remember: 'app/main.py is the product, custom_modes.yaml defines the responsible AI team, and handoff_controller.py verifies that the team moves forward only with the right evidence and a human approval.'